

Файлы в C++



- **Файл** – именованная область внешней памяти, выделенная для хранения массива данных. Данные, содержащиеся в файлах, имеют самый разнообразный характер: программы на алгоритмическом или машинном языке; исходные данные для работы программ или результаты выполнения программ; произвольные тексты; графические изображения и т. п.
- Файл имеет **имя** и **расширение**. Расширение позволяет идентифицировать, какие данные и в каком формате хранятся в файле.

Сигнатура файла

- **Сигнатура файла** - это первые байты, указывающие на тип файла.

Например:

MZP -это exe, dll;

BM - bmp

GIF89 - gif

Работа с файлами

Под работой с файлами подразумевается:

- создание файлов;
- удаление файлов;
- чтение данных;
- запись данных;
- изменение параметров файла (имя, расширение, атрибуты доступа и т.п.);
- другое.

ПОТОКОВЫЙ ВВОД-ВЫВОД В ФАЙЛЫ

- Для программиста открытый файл представляется как последовательность считываемых или записываемых данных.
- При открытии файла с ним связывается ***поток ввода-вывода***.
- Выводимая информация записывается в поток, вводимая информация считывается из потока.
- Для работы с файлами необходимо подключить заголовочный файл `<fstream>`:

```
#include <fstream>
```

<fstream>

- `ifstream` - файловый ввод ;
- `ofstream` - файловый вывод.
- Файловый ввод-вывод аналогичен стандартному вводу-выводу, единственное отличие — это то, что ввод-вывод выполнятся не на экран, а в файл.

При работе с файлом можно выделить следующие этапы:

1. создать объект класса `fstream` (`ofstream` или `ifstream`);
2. связать объект класса `fstream` с файлом, который будет использоваться для операций ввода-вывода;
3. осуществить операции ввода-вывода в файл;
4. **заккрыть файл.**

Пример

```
#include <fstream>
using namespace std;
int main()
{
    ofstream fout;
    fout.open("file.txt");
    fout << "Привет, мир!";
    fout.close();
    return 0;
}
```


Предупреждения

- Если файл с указанным именем существовал до запуска программы, то **после запуска программы все данные, что хранил существующий файл, будут уничтожены!**
- Если создается файл внутри каких-либо папок, то все эти папки должны уже существовать, иначе файл создан не будет.

Режимы открытия файлов

Константа	Описание
<code>ios::in</code>	открыть файл для чтения
<code>ios::out</code>	открыть файл для записи
<code>ios::ate</code>	при открытии переместить указатель в конец файла
<code>ios::app</code>	открыть файл для записи в конец файла
<code>ios::trunc</code>	удалить содержимое файла, если он существует
<code>ios::binary</code>	открытие файла в двоичном режиме

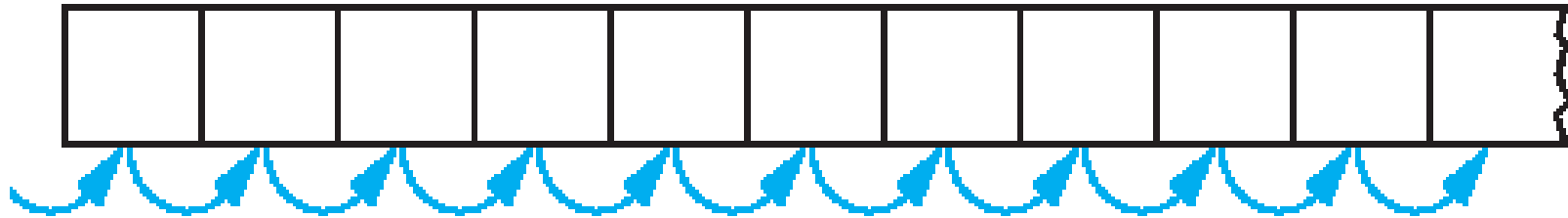
Режимы открытия файлов

- Режимы открытия файлов можно устанавливать непосредственно при создании объекта или при вызове метода `open()`.
- `ofstream fout("file.txt", ios::app);`
- `fout.open("file.txt", ios::app);`
- Режимы открытия файлов можно комбинировать с помощью поразрядной логической операции **ИЛИ**, например:

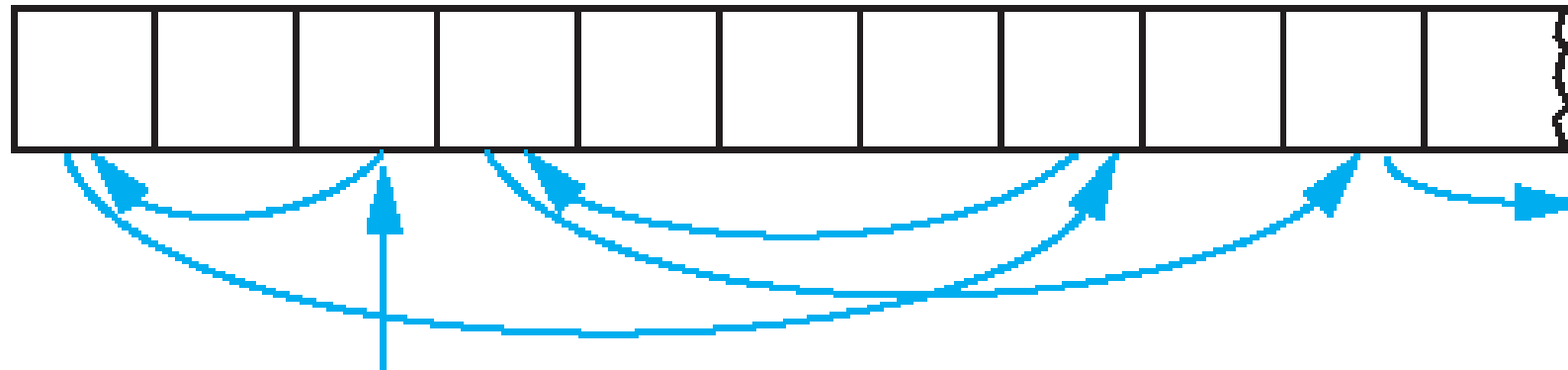
`ios::out | ios::in` - открытие файла для записи и чтения.

Доступ к файлам

- Последовательный



- Произвольный



Произвольный доступ к файлу

- Система ввода-вывода C++ позволяет осуществлять произвольный доступ с использованием методов `seekg()` и `seekp()`.
- `istream &seekg(Смещение, Позиция);`
- `ostream &seekp(Смещение, Позиция);`
- *Смещение* определяет область значений в пределах файла (тип `long int`).

Система ввода-вывода C++ обрабатывает два указателя, ассоциированные с каждым файлом:

- `get pointer g` - определяет, где именно в файле будет производиться следующая операция ввода;
- `put pointer p` - определяет, где именно в файле будет производиться следующая операция вывода.

Позиция смещения определяется как

Позиция	Значение
<code>ios::beg</code>	Начало файла
<code>ios::cur</code>	Текущее положение
<code>ios::end</code>	Конец файла

- Всякий раз, когда осуществляются операции ввода или вывода, соответствующий указатель автоматически перемещается.

С помощью методов **seekg()** и **seekp()** можно получить доступ к файлу в произвольном месте.

Можно определить текущую позицию файлового указателя, используя следующие функции:

- `streampos tellg()` - позиция для ввода
- `streampos tellp()` - позиция для вывода

Бинарные файлы

- Двоичный (бинарный) файл — в широком смысле: последовательность произвольных байтов.
- Название связано с тем, что байты состоят из бит, то есть двоичных (binary) цифр.
- В целом данный термин представляет собой меру отношения потребителя бинарного файла и самого файла.
- Если пользователь знает структуру и правила, по которым он способен преобразовать данный файл к более высокоуровневому, то он не является для него бинарным.
- Например, исполняемые файлы являются бинарными для пользователя компьютера, но при этом не являются таковыми для операционной системы.

Hex Editor Neo

File Edit View Select Operations Bookmarks NTFS Streams Tools History Window Help

English

New Document 1 copyy.png

00000000	00	01	02	03	04	05	06	07	08	09	0a	0b	0c	0d	0e	0f	
00000000	89	50	4e	47	0d	0a	1a	0a	00	00	00	0d	49	48	44	52	PNG.....
00000010	00	00	01	c5	00	00	02	17	08	02	00	00	00	62	bd	d2	...Ã.....
00000020	6a	00	00	00	01	73	52	47	42	00	ae	ce	1c	e9	00	00	j....sRGB.e
00000030	00	04	67	41	4d	41	00	00	b1	8f	0b	fc	61	05	00	00	..gAMA...±.
00000040	00	09	70	48	59	73	00	00	0e	c3	00	00	0e	c3	01	c7	..pHYs...Ã.
00000050	6f	a8	64	00	00	e4	47	49	44	41	54	78	5e	ec	fd	09	o`d..äGIDA
00000060	58	13	e7	de	f0	8f	9f	9e	3e	e7	9c	67	fb	fd	9f	7d	X.çBð Ÿž>ç
00000070	7b	9f	f7	7a	c7	ba	b6	b5	b6	b5	5a	3d	ad	5d	6c	b5	{Ÿ÷zÇ°pŸpŸ
00000080	ed	88	22	82	22	58	71	45	c1	15	37	70	c1	61	df	f7	í^", "XqEÁ.
00000090	1d	51	10	41	64	df	11	64	11	04	11	d9	77	64	91	1d	.Q.Adß.d...
000000a0	c2	16	20	0b	81	24	84	64	ae	fc	ef	99	84	3d	8d	41	Ã. . \$„døü
000000b0	25	83	cd	fd	b9	be	d7	65	e6	ce	64	48	be	33	f3	99	%fíý¹¼×eæï
000000c0	ef	3d	33	ce	fd	07	31	04	02	81	40	de	06	d0	a7	10	ï=3îý.1..
000000d0	08	04	f2	76	80	3e	85	40	20	90	b7	03	f4	29	04	02	..òv€>...@
000000e0	81	bc	1d	e6	fa	b4	ac	ac	2c	3a	3a	3a	0e	02	81	40	¼.æú´¬, ::
000000f0	20	af	22	3d	3d	5d	aa	4e	92	b9	3e	d5	d3	d3	fb	03	¬"==]²N'²>
00000100	04	02	81	40	14	43	aa	4e	12	d9	3e	1d	81	bc	fb	9c	.. @.C²N.Û>
00000110	3a	75	0a	ae	ca	f9	5c	bb	76	0d	a6	45	41	aa	ab	ab	:u.øÊù\»v.

Ready Offset: 00000000 (0) Size: 0x0000e4b2 (58,546): 57.17 KB Hex bytes, 16, Default ANSI OVR

History

Default

Open

File Attributes

Attribute	Value
File Name	C:\Users

File Attributes Selection

- Чтобы записать какое-нибудь значение в бинарном представлении, нужно для начала вывести это бинарное представление, а чтобы записалось правильное количество байт, **нужно явно указывать это количество.**
- Это выглядит приблизительно следующим образом:
- `(char*)&x` - делаем строку байтов для того, чтобы отдать потоку, открытому в двоичном режиме
- `sizeof(x)` - ограничиваем количество уходящих в поток байтов нужным числом
- Здесь есть аналогия с символьным массивом, но поскольку у нас не символьная строка, а **байтовая строка**, то ограничение происходит не нуль-символом, а явным указанием числа байт.

Кое-что об исключениях в C++

- При работе программ возникают так называемые *исключительные ситуации*, когда дальнейшее нормальное выполнение приложения становится невозможным.
- Причиной исключительных ситуаций могут быть как ошибки в программе, так и неправильные действия пользователя, неверные данные и т.д.
Обычные конструкции, необходимые для проверки данных, делают более-менее серьезную программу сложно читабельной.
- Программисту очень сложно отследить все исключительные ситуации.

блок try-catch

`try`

`{операторы защищенного блока}`

`catch(...)`

`{обработчик ошибочной ситуации}`

Важно! Многоточие является частью синтаксиса языка!

Пример

```
try {  
in.read( (char*) &S,  sizeof(S) );  
in.close();  
}  
catch (...) {  
std::cout << "File reading error!" << std::endl;  
}
```